SESSION 21 · DEVOPS CAPSTONE

15 October 2026

ProjectPulse

A delivery-board application, built and shipped the DevOps way — from a FastAPI endpoint to a monitored, autoscaled Kubernetes deployment.

React · FastAPI · PostgreSQL

Docker · Kubernetes · Helm

Prometheus · Grafana

WHAT IT IS

A delivery board, built to be judged on the pipeline — not just the app

The app

React + Vite dashboard, FastAPI + SQLAlchemy backend, PostgreSQL database. Create, filter, and track tasks through a responsive UI with loading and error states.

The point

Everything built around it: tests, containers, a CI/CD pipeline with a real security gate, Terraform, Kubernetes, Helm, Ingress, autoscaling, and live observability.

Not a copy

The course reference app is the instructor's TaskBoard. ProjectPulse follows the same DevOps path but is its own domain, UI, API, and code.

ARCHITECTURE — THE APP

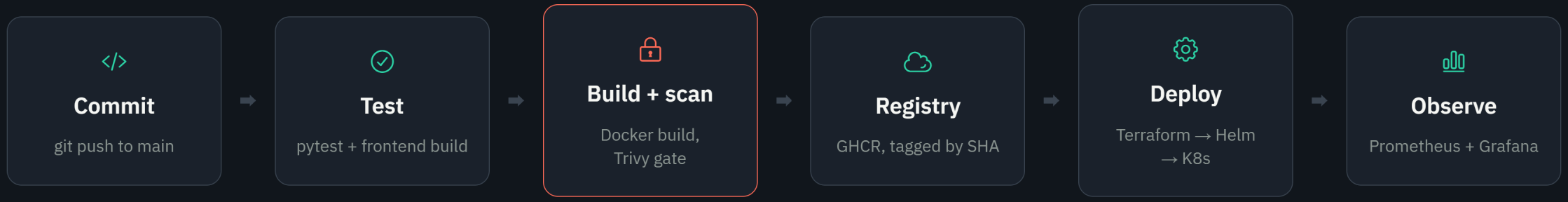
One request, four hops



Kubernetes needs a cheap liveness check and a real readiness check — that is exactly what **/health** and **/ready** are for; **/ready** actually verifies the database connection before answering.

ARCHITECTURE — THE PIPELINE

From a git push to a monitored cluster



Six stages, one pipeline. The red step is the one that actually failed a real build during this project — more on that shortly.

Ten endpoints: seven for tasks, three for the platform

Method	Path	Purpose
GET	/api/tasks	List and filter tasks
GET	/api/tasks/{id}	Read one task
POST	/api/tasks	Create a task
PUT	/api/tasks/{id}	Update a task
DELETE	/api/tasks/{id}	Remove a task
GET	/api/tasks/stats	Task counts by status
GET	/health	Liveness probe
GET	/ready	Readiness probe — checks the database
GET	/metrics	Prometheus-formatted metrics

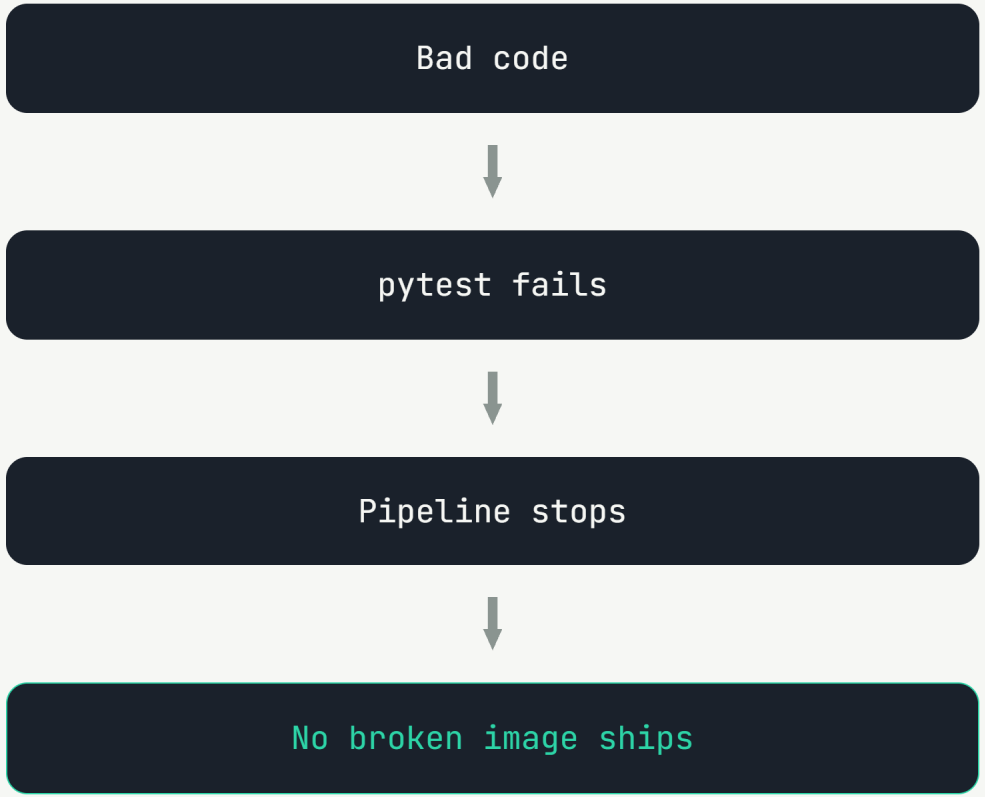
TESTING · M2

Tests are the first quality gate

6 pytest cases, isolated SQLite DB

- Health and readiness
- Create and read a task
- List and filter tasks
- Update a task
- Stats and delete
- Input validation

`pytest -v → 6 passed`

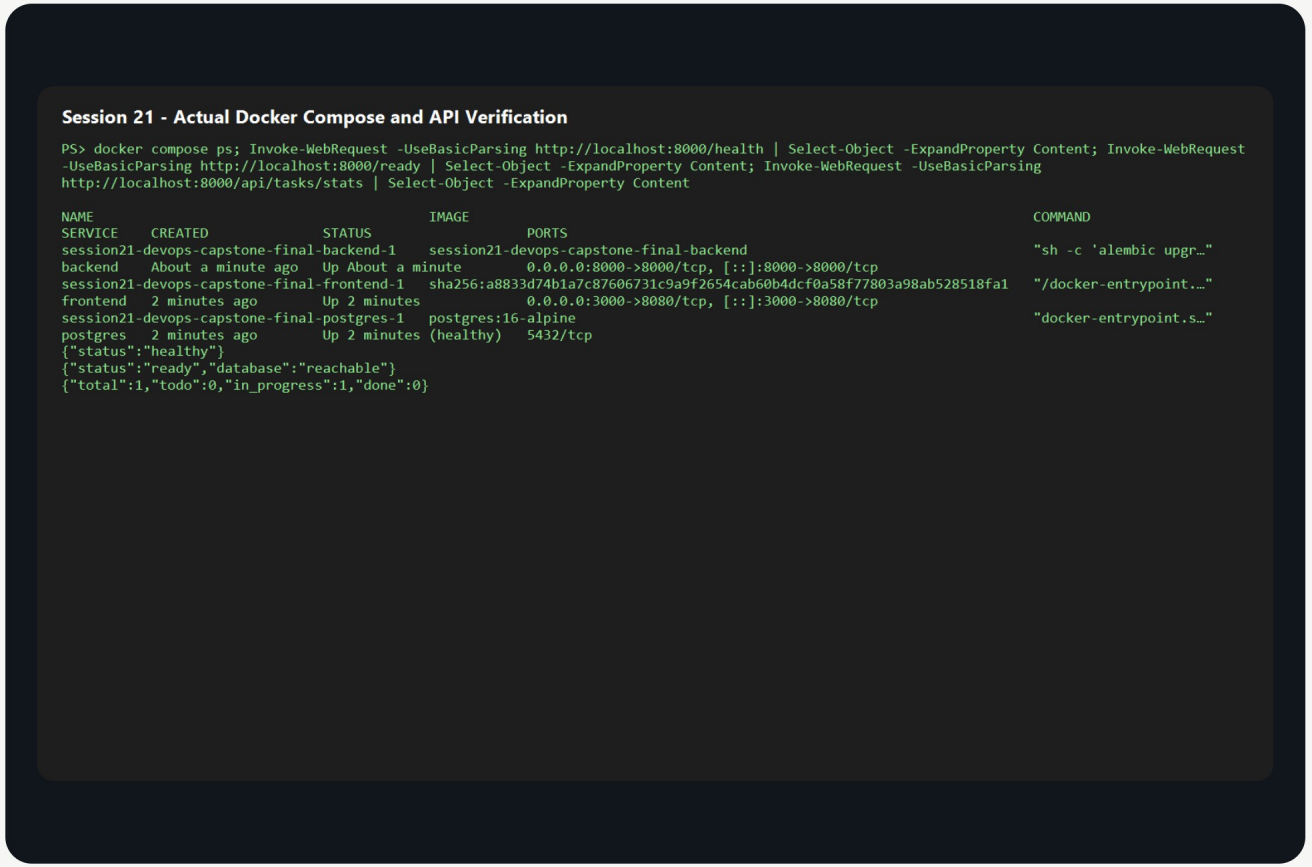


One command, three containers

backend/Dockerfile
python:3.12-slim → non-root user → runs Alembic migrations, then Uvicorn

frontend/Dockerfile
Node build stage → separate nginx-unprivileged runtime stage — no build tooling ships

```
docker compose up --build
```



Every change is a checkpoint, not a guess

Term	What it means here
git	Local version control — every change is a committed, recoverable snapshot
GitHub	The remote: collaboration, history, and the trigger for CI/CD
commit	An immutable checkpoint of the project at one moment
branch	An isolated line of development
pull request	Controlled, reviewable change — not a direct push into the unknown

```
public repository · amruthatavva2/devops-tasks
```

```
.gitignore excludes .venv, node_modules, __pycache__, *.db, Terraform state
```

CI/CD · M5

Two jobs, triggered on every push to main

`job: test-and-build`

- checkout + setup Python 3.12
- pytest backend/tests -v
- setup Node 22, npm ci, npm run build
- docker build × 2 (backend, frontend)
- Trivy scan × 2 — HIGH/CRITICAL fails the build

`job: publish-ghcr`

Runs only if test-and-build succeeds. Logs in with the repository's own GITHUB_TOKEN and pushes both images to GHCR.

```
commit A → image A
commit B → image B
```

Every image is tagged with the exact commit SHA — traceable straight back to source.

DEVSECOPS · M6 — THIS ACTUALLY HAPPENED

The security gate didn't just run — it stopped a real build

FAILED — first real run

Trivy scanned the backend image and found **3 HIGH CVEs** in **starlette** (a FastAPI dependency):

CVE-2025-62727
CVE-2026-48818
CVE-2026-54283

exit-code: 1 — the pipeline stopped. Frontend scan and GHCR publish were correctly skipped. The gate was not bypassed.

deps + base image upgrade

FIXED — green rerun

Upgraded fastapi, starlette, and the metrics library; patched the frontend's Alpine base packages.

0 HIGH/CRITICAL findings
6/6 pytest still passing
images pushed to GHCR

Verified by re-scan before it was ever pushed — not assumed.

REGISTRY · M5

Both images, live in GHCR

```
ghcr.io/amruthatavva2/projectpulse-backend:deb95abf3baf0ac6a1950d51513607d61bd671ff
```

```
ghcr.io/amruthatavva2/projectpulse-frontend:deb95abf3baf0ac6a1950d51513607d61bd671ff
```

Published by the **publish-ghcr** job after the fixed, fully green pipeline run — authenticated with the repository's own built-in **GITHUB_TOKEN**, no manual login.

INFRASTRUCTURE AS CODE · M7

A real AWS VPC and EKS cluster, described in code

What terraform/ builds

- VPC with 2 public subnets
- Internet Gateway + routes
- EKS control plane
- IAM roles for the cluster and the nodes
- A managed EKS node group

Why it isn't applied today

EKS can't be honestly emulated locally. Running this for real needs an AWS account, credentials, and acceptance of real cost — a decision, not an oversight.

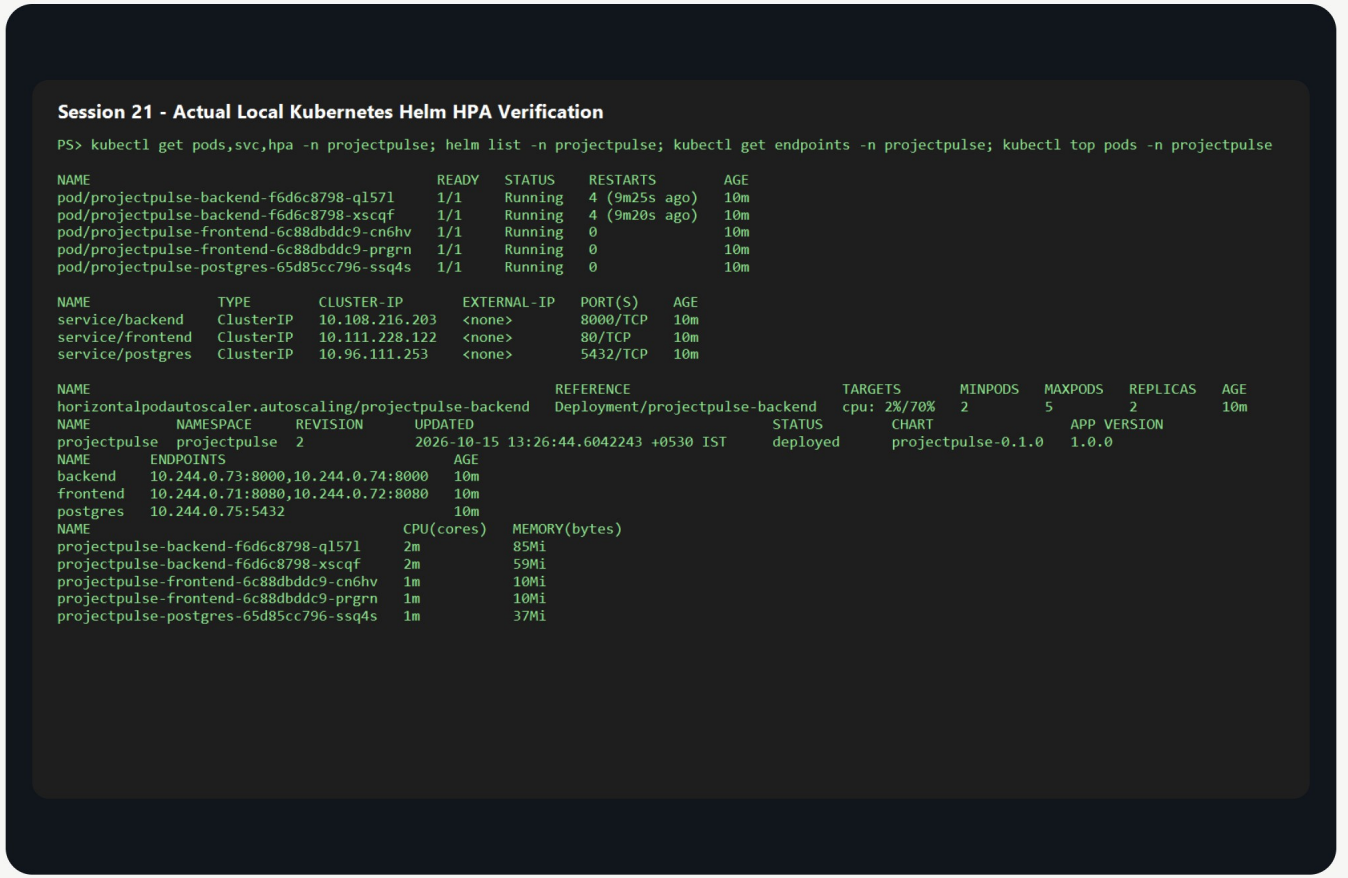
```
terraform fmt & validate — pass
terraform init / plan / apply — ready, not run
```

KUBERNETES + HELM · M8

One Helm chart, every Pod Running

helm upgrade --install deploys the namespace, two Deployments, three Services, the backend HPA, and a PVC-backed Postgres — in one command.

2 frontend + 2 backend replicas · 3 ClusterIP Services · HPA 2–5 · live endpoints



INGRESS + HPA · M8

Routed by the controller, scaled by CPU — both proven live

Ingress — verified through the controller

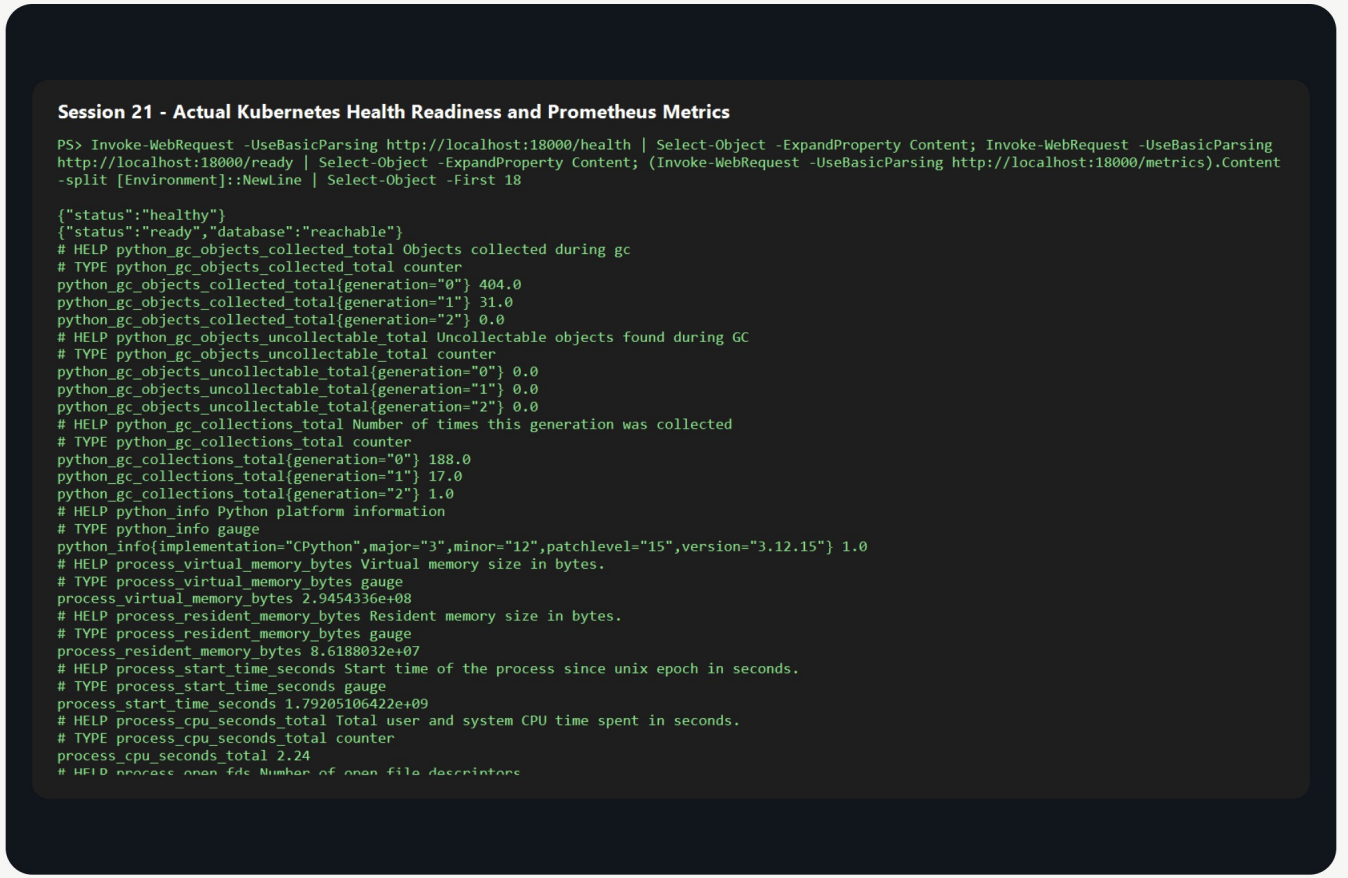
/ → frontend, HTTP 200

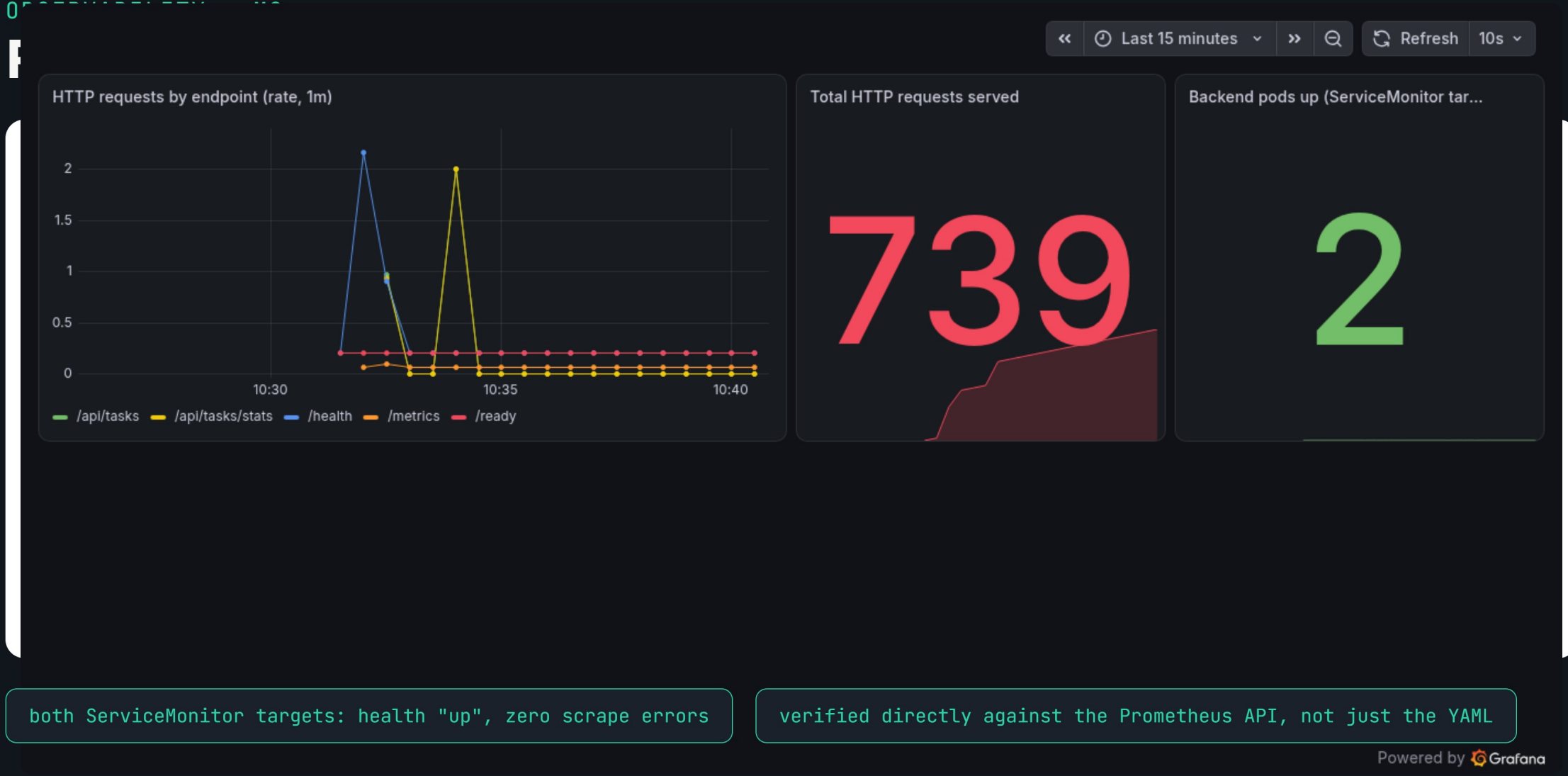
/api/tasks/stats → backend, valid JSON

HPA — CPU-based, 2 to 5 replicas

scaleTargetRef: backend Deployment

target: 70% average CPU utilization





both ServiceMonitor targets: health "up", zero scrape errors

verified directly against the Prometheus API, not just the YAML

LIVE DEMO

Two broken deployments, diagnosed on stage

broken-image.yaml

A Deployment pointing at a tag that does not exist.

```
kubectl get pods
kubectl describe pod <name>
kubectl get events --sort-by=.lastTimestamp
```

→ ImagePullBackOff, fixed by correcting the tag

broken-service.yaml

A Service whose selector matches no Pod's labels.

```
kubectl get endpoints broken-selector
kubectl get pods --show-labels
```

→ empty endpoints, fixed by correcting the selector

The lesson behind both: a Service routes by label match, and Kubernetes will tell you exactly what's wrong — if you know where to look.

THE FULL CHECKLIST

9 of 10 modules done and verified live

M	Module	Status
M1	Application — frontend + backend + database	✓ verified live
M2	Testing — pytest + quality	✓ 6/6 passing
M3	Git and GitHub	✓ done
M4	Docker — Dockerfile + Compose	✓ verified live
M5	CI/CD — GitHub Actions pipeline	✓ green run, GHCR published
M6	DevSecOps — Trivy security scan	✓ caught a real CVE, then fixed
M7	Terraform — AWS infrastructure	code ready — not applied (real cost)
M8	Kubernetes + Helm	✓ verified live, incl. Ingress + HPA
M9	Observability — Prometheus + Grafana	✓ target UP, dashboard populated
M10	Documentation + presentation	✓ this deck + full README

Thank you

ProjectPulse — built, tested, containerized, scanned, deployed, and watched, the same way a real team would do it.

github.com/amruthatavva2/devops-tasks

Questions?